



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

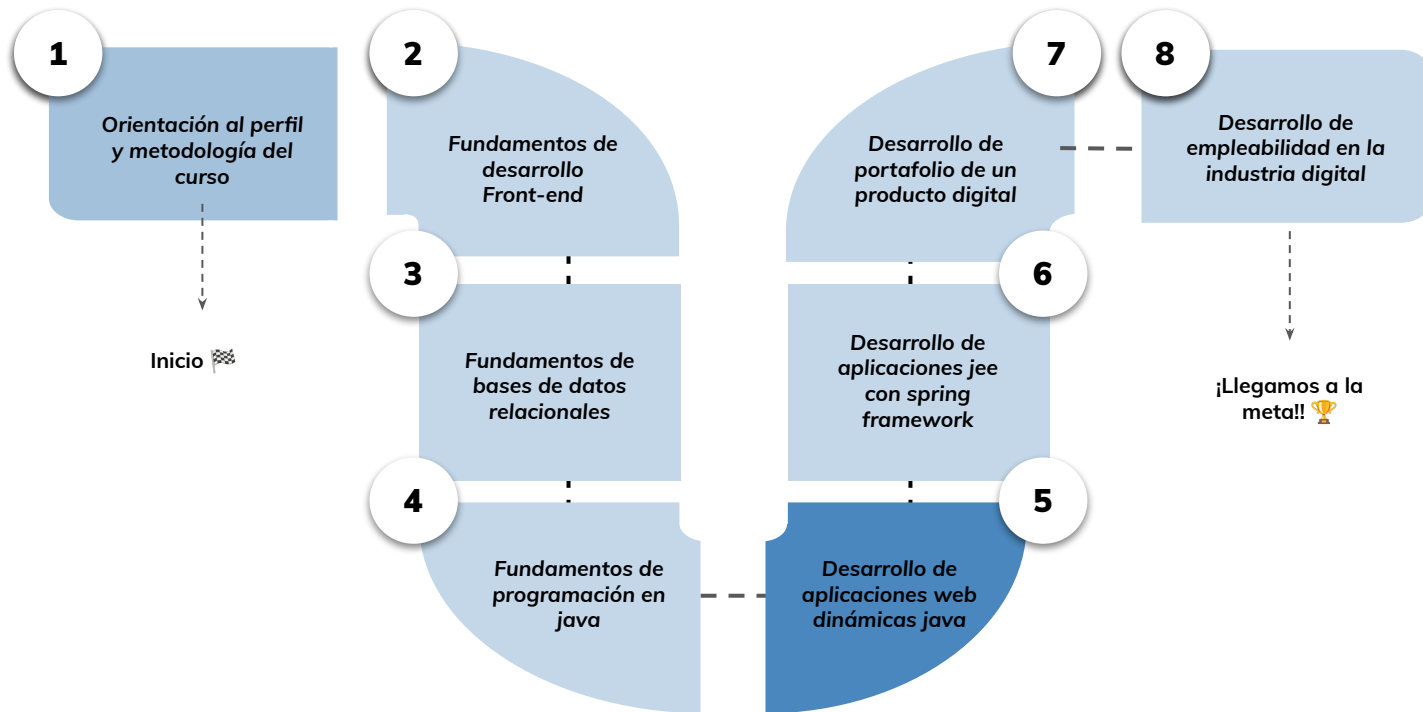


El patrón de diseño MVC - › Parte 1

Aprendizaje Esperado 5: Aplicar patrón modelo-vista- controlador en una aplicación web dinámica para la implementación de un flujo de navegación que despliega y captura información de acuerdo a lo solicitado.

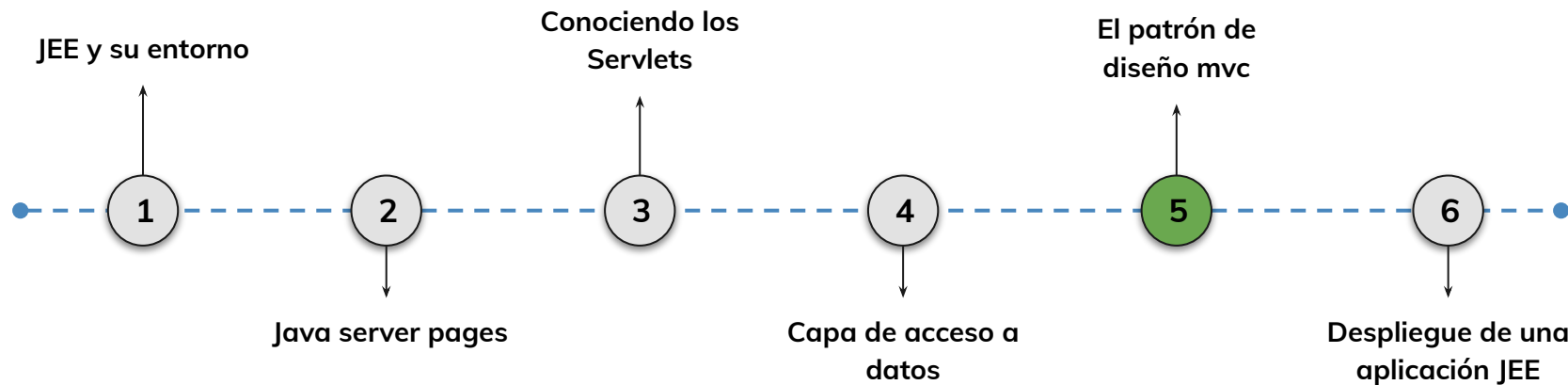
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Implementación del patrón
Modelo-Vista-Controlador
(MVC)

Trabajaremos en comprender
y aplicar el patrón de diseño
MVC en una aplicación web
dinámica, permitiendo la
separación de lógica,
presentación y flujo de control.

Introducción al patrón MVC

¿Qué es un patrón de diseño?

Elementos y beneficios del
patrón MVC

Estructura práctica del
patrón MVC en Java

Componentes de Modelo, Vista
y Controlador

Ejemplo práctico MVC con Java
EE



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender el concepto de patrón de diseño y su utilidad.
- Identificar las partes que componen el patrón MVC.
- Aplicar el patrón MVC para separar la lógica de negocio, presentación y control.
- Construir clases modelo, vistas JSP y controladores en Java.
- Implementar flujos de navegación dinámicos que capturan y despliegan información

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Obtuvimos datos de una base de datos utilizando `ResultSet`.
- Modificamos registros en la base de datos mediante `JDBC`.
- Conocimos la estructura y el propósito de la capa de acceso a datos (DAL).
- Implementamos una clase DAO para gestionar la interacción con la base de datos.
- Conocimos el patrón DTO y su utilidad para transferir datos entre capas.

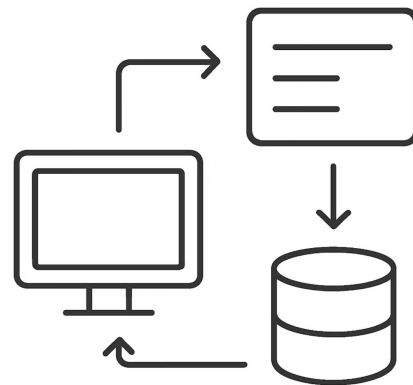
➤ El patrón de diseño MVC



El patrón de diseño MVC

El desarrollo de aplicaciones web ha evolucionado adoptando patrones arquitectónicos que permiten crear sistemas escalables y mantenibles.

Uno de los más utilizados es el patrón **Modelo-Vista-Controlador (MVC)**, que promueve la separación de conceptos y facilita el desarrollo de interfaces de usuario ricas.





¿Qué es un patrón de diseño?

Un patrón de diseño es una **solución reutilizable y general para un problema recurrente en el desarrollo de software**. Los patrones de diseño proveen **templates predefinidos para resolver situaciones comunes** en la construcción de aplicaciones, permitiendo resolver estos problemas de manera eficiente y probada.

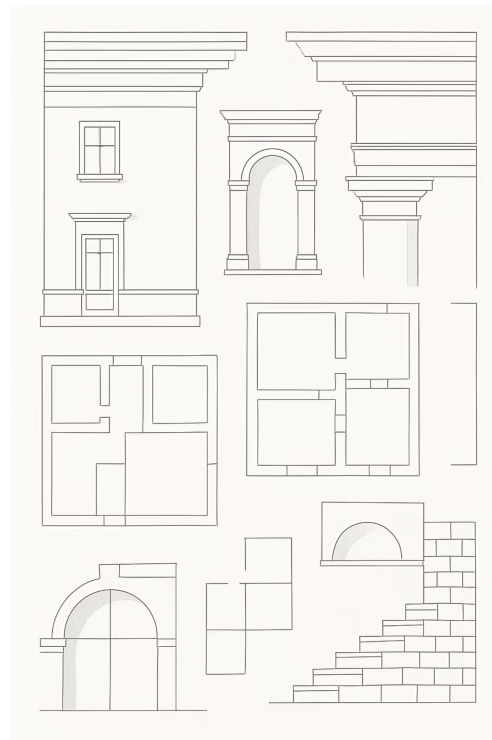




Origen de los patrones de diseño

Los patrones de diseño surgieron a partir de la necesidad de contar con soluciones estandarizadas para problemas habituales como creación de objetos, estructuración de clases e interfaces o relaciones entre componentes.

En lugar de resolver estas cuestiones una y otra vez, los patrones capturan las mejores prácticas en formatos reutilizables.





Características de los patrones de diseño



Reutilizables

Son reutilizables en distintos contextos y lenguajes de programación.



Soluciones a problemas recurrentes

Resuelven problemas recurrentes en el desarrollo de software.



Buenas prácticas

Facilitan el uso de buenas prácticas probadas y estandarizadas.



Orientación a objetos

Se basan en principios de diseño orientado a objetos.



Mejoran el código

Mejoran la legibilidad y mantenimiento del código.

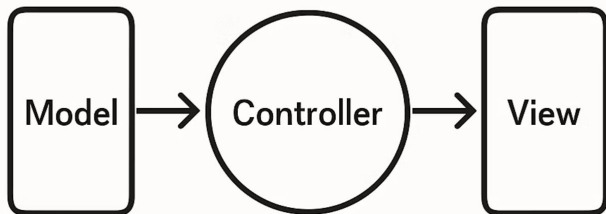




Patrones de diseño comunes

En resumen, los patrones de diseño como MVC, Singleton, Observer y otros tantos, nos permiten resolver de forma óptima y reutilizable problemas comunes de diseño en nuestras aplicaciones.





Patrón MVC: Definición

MVC (Modelo-Vista-Controlador) es un patrón en el diseño de software comúnmente utilizado para implementar interfaces de usuario, datos y lógica de control. Enfatiza una **separación entre la lógica de negocios y su visualización**. Esta "separación de preocupaciones" proporciona una mejor división del trabajo y una mejora de mantenimiento.





Las tres partes del MVC





Beneficios de utilizar MVC

Utilizar el patrón MVC en aplicaciones web aporta varios beneficios:

- Permite separar la lógica de negocio de la interfaz de usuario, resultando en una mejor organización del código.
- Promueve la reutilización de código, por ejemplo se puede usar el mismo modelo para distintas interfaces.
- Facilita el mantenimiento y actualización, ya que se pueden modificar el modelo, vista o controlador de forma independiente.





Más beneficios del patrón MVC

División por roles

El desarrollo se puede dividir por roles: programadores de backend en el modelo, diseñadores frontend en la vista, etc.

Menor impacto de cambios

Los cambios tienen menor impacto al estar desacoplados en capas separadas.

Escalabilidad independiente

Permite escalar cada componente de forma independiente.





Ventajas adicionales del MVC

Pruebas unitarias

Es más fácil realizar pruebas unitarias enfocadas en cada capa.

Abstracción visual

La interfaz visual queda abstracta del modelo de datos subyacente.

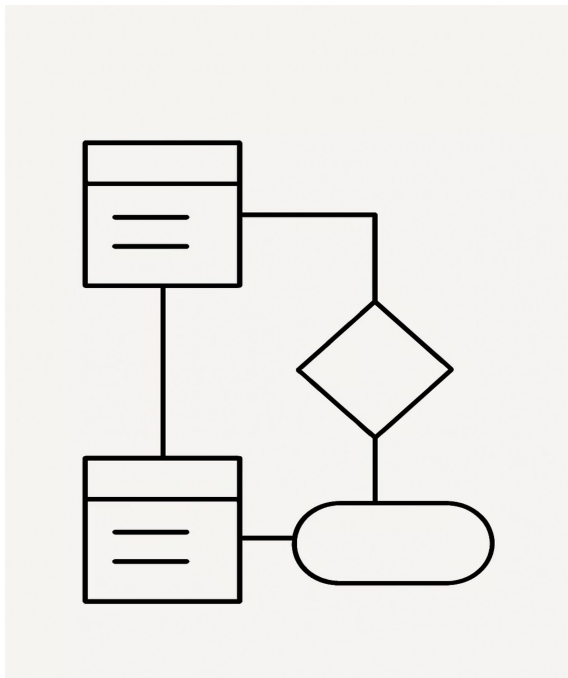
Acoplamiento débil

Promueve un acoplamiento más débil entre los componentes.

Código mantenible

Siguiendo estándares resulta en un código más claro y mantenible.





Elementos del Modelo

En una aplicación web dinámica desarrollada con Java, los elementos típicos que pertenecen a la capa de Modelo en el patrón MVC son:





Clases del Modelo

Clases POJO

Clases POJO (**P**lain **O**ld **J**ava **O**bjects) que representen **entidades** y objetos de negocio. Por ejemplo, clases Usuario, Producto, Factura, etc.

Objetos de acceso a datos

Objetos de acceso a datos, como DAOs (Data Access Objects), para abstraer la lógica de **persistencia**.





Lógica de negocio en el Modelo

- Clases que implementen la lógica de negocio más allá de los datos, como cálculos, algoritmos, validaciones, etc.
- Objetos de dominio que modelen conceptos específicos del negocio.
- Conexiones a sistemas externos como bases de datos o APIs.





Componentes adicionales del Modelo

Librerías y frameworks

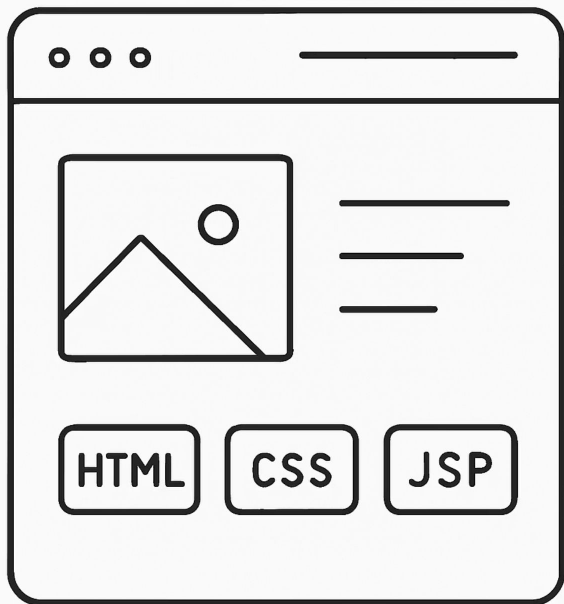
Librerías y frameworks reusables enfocados en la capa de negocio.

Servicios

Interfaces y clases de **servicios** que abstraiga funcionalidades de negocio.

En resumen, el **modelo contiene toda la lógica de negocio y datos**, independiente de la visualización y la lógica de control. Solo se enfoca en encapsular las reglas y objetos de dominio.





Elementos de la Vista

En el patrón MVC, los elementos típicos que pertenecen a la capa de Vista en una aplicación web dinámica con Java son:



Componentes principales de la Vista

1

Páginas JSP

Páginas **JSP** que definen la interfaz de usuario. Contienen HTML, CSS y código JSP para dinamismo.

2

Archivos estáticos

Archivos **estáticos** como imágenes, CSS y JavaScript utilizados para la presentación.

3

Componentes visuales

Componentes visuales reusables como custom tags o macros.





Elementos adicionales de la Vista

1

Layouts y templates

Layouts y templates para estructurar contenidos comunes.

2

Librerías JSTL

Librerías de etiquetas **JSTL** para facilitar la visualización de datos.

3

Internacionalización

Internacionalización mediante archivos de propiedades.

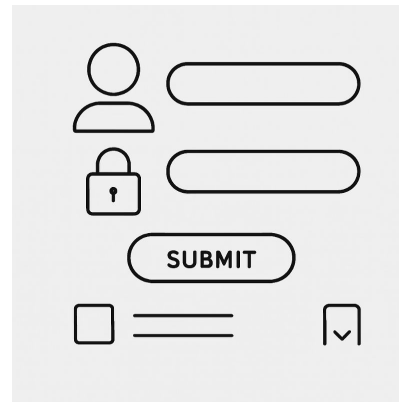


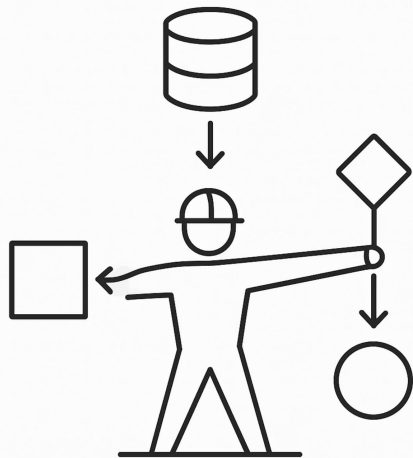


Componentes finales de la Vista

- Formularios para recolectar inputs del usuario.
- Clases que implementen funcionalidades de renderizado.

En resumen, la capa de **Vista** contiene todo lo relacionado a la **interfaz de usuario y la presentación visual** de la información, separado de la lógica de negocios.





Elementos del Controlador

Los principales elementos que pertenecen a la capa de Controlador en aplicaciones web MVC con Java son:



Componentes principales del Controlador

Servlets

Servlets que implementan la lógica de control y manejo de requests y responses.

Filtros

Filtros (filters) para interceptar y modificar requests/responses.

Clases controladoras

Clases controladoras Front Controller cuando se utiliza ese patrón.





Elementos adicionales del Controlador

Acciones específicas

Acciones específicas para manejar lógica de control de ciertas peticiones.

APIs de mapeo

APIs de mapeo de URLs a métodos como en los frameworks MVC.

Objetos de workflow

Objetos que implementan workflow y navegación de la aplicación.



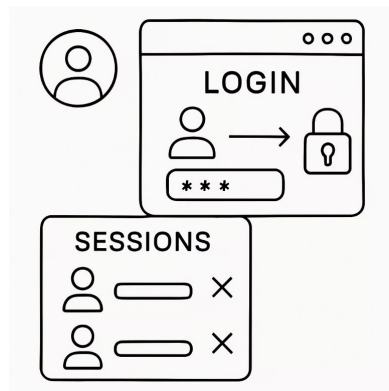


Componentes finales del Controlador

Control de **sesión y autenticación** de usuarios.

- Transiciones y redireccionamiento según estado de la aplicación.
- Excepciones relacionadas al control de flujo.

El **controlador gestiona el flujo de la aplicación**, delegando trabajo al modelo y las vistas según corresponda.





Flujo de trabajo en MVC



Usuario

El usuario interactúa con la interfaz (Vista)



Controlador

El controlador recibe la petición y la procesa



Modelo

El modelo actualiza o consulta los datos



Vista

La vista muestra el resultado al usuario

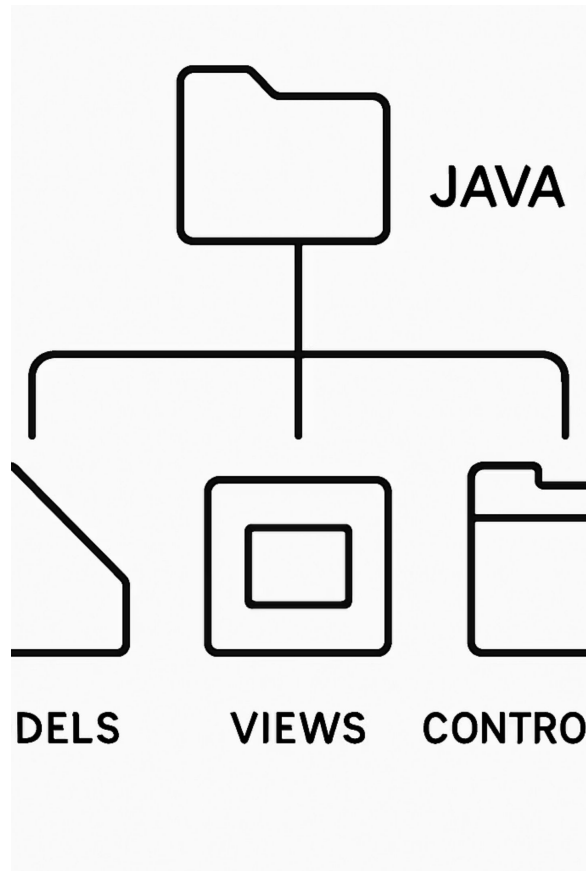




Ejemplo de MVC en Java EE

Estructura de proyecto

- Modelo: Clases Java (POJOs, DAOs, Services)
- Vista: Archivos JSP, HTML, CSS
- Controlador: Servlets, Filtros



< > Ejemplo de Modelo en Java

```
// Clase POJO (Entidad)
public class Usuario {
    private int id;
    private String nombre;
    private String email;

    // Getters y setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    // ...
}
```



< > Ejemplo de DAO en Java

```
// Clase DAO (Acceso a datos)
public class UsuarioDAO {
    public Usuario obtenerPorId(int id) {
        // Lógica para obtener usuario de la base de datos
        return usuario;
    }

    public void guardar(Usuario usuario) {
        // Lógica para guardar usuario en la base de datos
    }
    // ...
}
```



< > Ejemplo de Servicio en Java

```
// Clase de Servicio (Lógica de negocio)
public class UsuarioService {
    private UsuarioDAO usuarioDAO = new UsuarioDAO();

    public boolean autenticar(String email, String password) {
        Usuario usuario = usuarioDAO.obtenerPorEmail(email);
        // Lógica de validación de credenciales
        return esValido;
    }
    // ...
}
```



< > Ejemplo de Vista en JSP

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
<title>Lista de Usuarios</title>
</head>
<body>
<h1>Usuarios Registrados</h1>
<table>
<c:forEach items="${usuarios}" var="usuario">
<tr>
<td>${usuario.nombre}</td>
<td>${usuario.email}</td>
</tr>
</c:forEach>
</table>
</body>
</html>
```



< > Ejemplo de Controlador en Java

```
// Servlet (Controlador)
@WebServlet("/usuarios")
public class UsuarioServlet extends HttpServlet {
    private UsuarioService usuarioService = new UsuarioService();

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        // Obtener datos del modelo
        List usuarios = usuarioService.listarTodos();

        // Pasar datos a la vista
        request.setAttribute("usuarios", usuarios);

        // Redirigir a la vista
        request.getRequestDispatcher("/WEB-INF/usuarios.jsp")
            .forward(request, response);
    }
    // ...
}
```



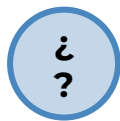
Live Coding

¿En qué consistirá la Demo?

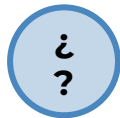
Durante esta demo construiremos una pequeña aplicación web Java EE que implementa el patrón Modelo-Vista-Controlador (MVC). El objetivo será desplegar una lista de usuarios desde una base de datos simulada, mostrando la separación clara entre Modelo (datos y lógica), Vista (interfaz) y Controlador (flujo).

1. Crear la clase POJO **Usuario**
2. Implementar el DAO **UsuarioDAO**
3. Diseñar el servicio **UsuarioService**
4. Crear el servlet **UsuarioServlet**
5. Diseñar la vista **usuarios.jsp**
6. Configurar el mapeo de URLs
7. Agregar archivos estáticos
8. Probar la aplicación

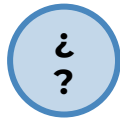
#Momentode Preguntas...



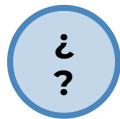
¿Qué ventajas aporta el patrón MVC frente a una arquitectura monolítica?



¿Qué responsabilidades tiene cada una de las capas: Modelo, Vista y Controlador?



¿Cómo se comunican entre sí las capas en una aplicación Java?



¿Por qué facilita el mantenimiento y testing una arquitectura basada en MVC?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Gestor de Usuarios MVC

Gestor de Usuarios MVC

Contexto: 🙌

Un equipo de desarrollo está trabajando en una aplicación web que debe mostrar la lista de usuarios registrados y permitir su visualización desde una interfaz amigable. Necesitan implementar el patrón MVC para lograr una separación clara entre la lógica de negocio, la interfaz gráfica y el flujo de navegación.

Consigna: 📝

Implementa una aplicación web básica en Java usando el patrón Modelo-Vista-Controlador. El sistema debe tener un servlet que gestione la solicitud de listado de usuarios, un modelo con POJO y DAO, y una vista en JSP que muestre la tabla de usuarios.

Tiempo 🕒: 45 minutos

Paso a paso: ⚙️

1. Crea una clase Usuario con campos id, nombre, email.
2. Implementa la clase UsuarioDAO con un método listarTodos().
3. Crea UsuarioService que llame a UsuarioDAO.
4. Desarrolla un servlet UsuarioServlet que invoque al servicio y redirija a una JSP.
5. Crea una vista usuarios.jsp que muestre el listado recibido.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Comprendimos el concepto de patrón de diseño
- ✓ Exploramos la estructura del patrón Modelo-Vista-Controlador
- ✓ Identificamos componentes del modelo, vista y controlador en Java
- ✓ Desarrollamos un ejemplo de implementación práctica de MVC
- ✓ Vimos beneficios como escalabilidad, mantenimiento y reutilización
- ✓ Aplicamos la lógica de separación de responsabilidades



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ———— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

